
Group MLR-21 WingA4_771 Minor Assignment 1

Nitin Maheshwari (230711)
nitinm23@iitk.ac.in

Vidit Agarwal (231148)
viditag23@iitk.ac.in

Jeetraj Gupta (230507)
jeetrajg23@iitk.ac.in

Aaryan Tiwari (230023)
aaryant23@iitk.ac.in

Yugan Jain (231202)
yuganjain23@iitk.ac.in

Harshit (230457)
jharshit23@iitk.ac.in

1 Problem Abstract

We have been provided with Melbo's buggy code for a CSVM solver, which only achieves a 30% accuracy in comparison to the general 95%+ benchmark.

After debugging the code, we have been provided with a set of questions which have to be answered.

2 Data Description

We are given data from an **arbiter-PUF** with **32-bit** challenges. There are 100,000 challenges and corresponding responses in the file. Of these, 1,000 data points will be randomly chosen to be used for training and the rest 99,000 will be used for testing.

3 Tasks

3.1 Problem 1:

Question: Discover all coding errors made by Melbo in implementing the SDCM algorithm. Recall that all coding errors are confined to cell number 5 of the Jupyter notebook. Describe the errors in your report along with a description of why you think it is an error. (10 marks)

Solution:

- Corrections in **doCoordOptCSVMDual** Function:

- Edit 1: Added α_i to α_i^{new}

$$\alpha_i^{new} = \frac{1 - y_i (x \cdot w_{SDCM} + b_{SDCM})}{\|x_i\|^2} + \alpha_i$$

- Edit 2: Changed sign from $<$ to $>$ in constraining α_i

```
if newAlphai > C:  
    newAlphai = C  
if newAlphai < 0:  
    newAlphai = 0
```

- Edit 3: Correcting the formula for updating w & b

In SDCM, after we obtain the value of α_i^{new} , we update w in accordance of delta between α_i^{new} and α_i . In the given code, we have just added them, which is incorrect.

Original Formula:

$$w_{SDCM} = w_{SDCM} - (\alpha_i + \alpha_i^{new}) \cdot y_i \cdot x$$

$$b_{\text{SDCM}} = b_{\text{SDCM}} - (\alpha_i + \alpha_i^{\text{new}}) \cdot y_i$$

Corrected Formula:

$$w_{\text{SDCM}} = w_{\text{SDCM}} - (\alpha_i - \alpha_i^{\text{new}}) \cdot y_i \cdot x$$

$$b_{\text{SDCM}} = b_{\text{SDCM}} - (\alpha_i - \alpha_i^{\text{new}}) \cdot y_i$$

- Created the not defined functions - `getCyclicCoord` & `getRandCoord`

```
def getCyclicCoord(state):
    idx = state[0]
    seq = state[1]

    d = len(seq)

    if idx >= d:
        idx = 0
    else:
        idx += 1

    state = (idx, seq)
    curr = idx
    return (curr, state)
```

```
def getRandCoord(state):
    idx = state[0]
    d = state[1]

    curr = np.random.randint(0, d - 1)
    state = (curr, d)
    return (curr, state)
```

3.2 Problem 2:

Question: Once you have corrected Melbo's mistakes, what test accuracy do you get? Mention this and show what primal and dual objective curve do you get. Do not change any hyperparameter values or settings (e.g. value of C, number of training points, initialization, horizon etc) in any cell other than cell number 5 while answering this question. (10 marks)

Accuracy = 95 - 97% on different re-runs.

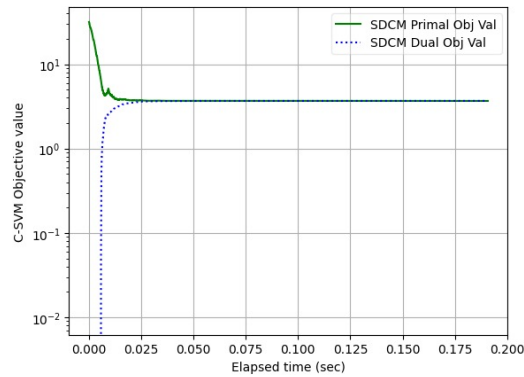


Figure 1: Objective Function plot for Primal and Dual

3.3 Problem 3:

Question: Change the value of C between very small values, e.g. 0.00001, and very large values, e.g. 10000, without changing any of the other hyperparameters/settings. What changes do you observe in the test accuracy? What changes do you observe in the primal-dual objective curve? Please do these experiments after you have fixed Melbo's errors. (10 marks)

Solution: When we change our C, the accuracy of the model remains more or less the same, which can be observed in the following table. However, for small values of C like 0.01, both objective

functions converge nicely in line with theory, but as we increase C , the functions converge later, and for large C , they don't converge at all.

C	Accuracy
10^{-5}	0.9374
10^{-4}	0.9373
10^{-3}	0.9381
10^{-2}	0.9585
10^{-1}	0.9705
1	0.9765
10	0.9721
100	0.9610
1000	0.9729
10000	0.9601

Table 1: Accuracy of SVM for different values of C

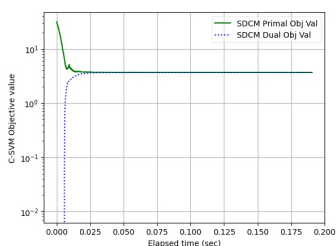


Figure 2: $C = 0.01$

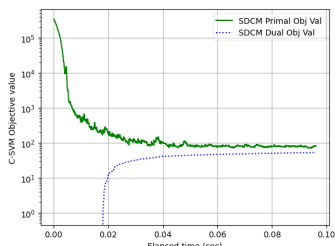


Figure 3: $C = 1$

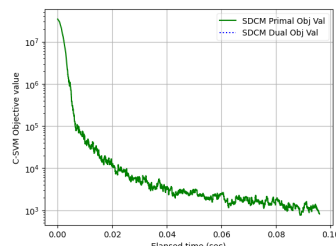


Figure 4: $C = 10$

Figure 5: Objective functions for different values of C

3.4 Problem 4:

Question: Experiment with at least two of the hyperparameters mentioned below and comment on their effect on test accuracy and primal-dual convergence curve. You must report the test accuracies and the convergence curves in your report after changing these hyperparameters. Please do these experiments after you have fixed Melbo's errors. (20 marks)

3.4.1 Part (a)

Question: Effect of changing the coordinate generator. Currently it is set to randperm. What happens if we use cyclic? random?

Solution: We experimented with three coordinate generators: randperm, cyclic, and random, while keeping all other hyperparameters fixed. The observed test accuracies were:

- randperm: 95.2%
- cyclic: 94.6%
- random: 95.0% (with slight variation across runs)

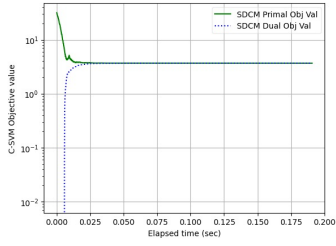


Figure 6: Randperm

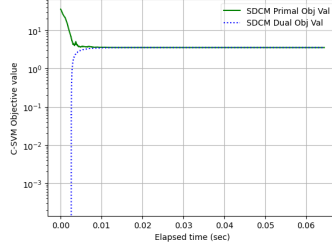


Figure 7: Cyclic

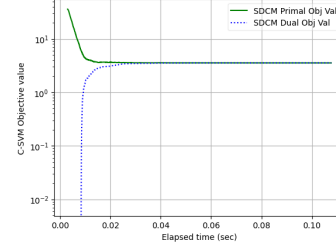


Figure 8: Random

Figure 9: Primal-dual objective convergence curves for different coordinate generators.

3.4.2 Part (b)

Question: Effect of changing the initialization `initDual`. Currently it is set to “all-C” values. What happens if we do an all zero initialization? Random initialization? You may use `numpy.random` submodule to generate random numbers.

Solution : Changing the initialisation does not affect the final accuracy or the final optimum, but it may affect how fast the convergence occurs.

3.4.3 Part (c)

Question: Effect of horizon (the number of iterations) – note that this is passed as an argument to the function `doSDCM`. Currently it is set to 20,000. Does increasing it by a lot get a much better solution? Does decreasing it a lot make the solution much worse?

Solution: Accuracy increases as we increase the number of iterations.

Horizon	Accuracy
20	0.9276
200	0.9309
2000	0.9511
20000	0.9529
200000	0.9528

Figure 10: Accuracy vs Horizon

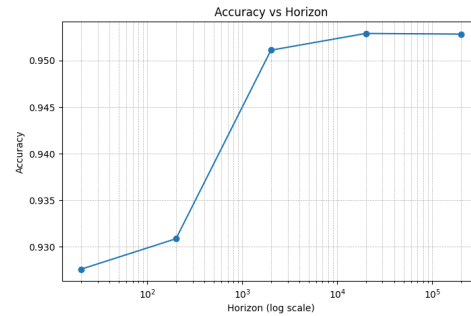


Figure 11: Horizon vs Accuracy

3.4.4 Part (d)

Question: Number of training points (currently set to 1000). Does increasing or decreasing the number of training points by a lot affect test accuracy and convergence? If so, how?

Solution: As we increase the amount of data we have available for training our model, we can find better parameters as our model has more data to learn from.

Training Size	Accuracy
10	0.61
100	0.854
1000	0.955
10000	0.980
50000	0.987

Figure 12: Accuracy vs Training Size

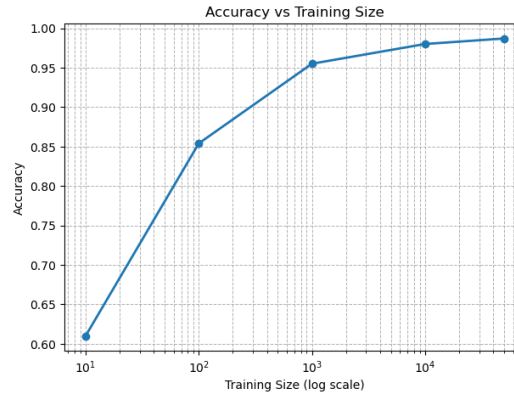


Figure 13: Accuracy vs Training Size

3.4.5 Part (e)

Question: Random chance – even if you keep all hyperparameters and settings fixed (say to their default values), does running the entire Jupyter notebook from first-to-last-cell again and again change the test accuracy? What could be the reason behind this?

Solution :Yes, the testing accuracy can change slightly even when all the hyperparameters are kept same, this happens because during the train-test split operation the seed value is not fixed in the code meaning that each time we run all the cells the splitting will be random, i.e there will be different puf challenges in the training set every time we run all the cells, so the model trains and test on different puf challenges every time we run all the cells, therefore we observe a variation in the test accuracy. Seeding refers to fixing the starting point of the pseudo-random number generator, if this seed is fixed then the random operation is reproducible and will give same result each time the code is executed

References

[1] Numpy Documentation